Lab 17 – AI for Data Processing

Name: M.Dinesh

Roll.no:2503A52374

Batch:13

Task 1 – Customer Data Cleaning

Task:
Use AI to generate a Python script for cleaning a **customer**

Prompt:

Generate a Python script using Pandas to clean the customer dataset. Handle missing values in age, income, and city, remove duplicate records, convert city names to lowercase, and encode gender and city categorical variables.

Code:

```
import pandas as pd

# Load dataset
data = pd.read_csv("shopping_trends.csv")

print("Original Dataset:")
print(data.head())

# Handle missing values
data['Age'] = data['Age'].fillna(data['Age'].median())

# Handle Income if available
if 'Income' in data.columns:
    data['Income'] = data['Income'].fillna(data['Income'].median())
```

```
# Use Location as City
if 'Location' in data.columns:
    data['Location'] = data['Location'].fillna(data['Location'].mode()[0])
    data['Location'] = data['Location'].str.lower().str.strip()

# Remove duplicate records
data = data.drop_duplicates()

# Encode categorical variables
data['Gender'] = data['Gender'].astype('category').cat.codes

if 'Location' in data.columns:
    data['Location'] = data['Location'].astype('category').cat.codes

# Display cleaned dataset
print("\nCleaned Dataset:")
print(data.head())

# Check missing values
print("\nMissing Values:")
print(data.isnull().sum())

# Save cleaned dataset
data.to_csv("cleaned_customer_data.csv", index=False)

print("\nData Cleaning Completed Successfully!")
```

Output:

```
Original Dataset:
   Customer ID  Age  ... Preferred Payment Method Frequency of Purchases
0            1   55  ...                   Venmo            Fortnightly
1            2   19  ...                    Cash            Fortnightly
2            3   50  ...             Credit Card                 Weekly
3            4   21  ...                  PayPal                 Weekly
4            5   45  ...                  PayPal                Annually

[5 rows x 19 columns]

Cleaned Dataset:
   Customer ID  Age  ...  Preferred Payment Method Frequency of Purchases
0            1   55  ...                   Venmo            Fortnightly
1            2   19  ...                    Cash            Fortnightly
2            3   50  ...             Credit Card                 Weekly
3            4   21  ...                  PayPal                 Weekly
4            5   45  ...                  PayPal                Annually

[5 rows x 19 columns]

Missing Values:
Customer ID                 0
Age                         0
Gender                      0
Item Purchased              0
Category                    0
Purchase Amount (USD)       0
Location                    0
Size                        0
Color                       0
Season                      0
Review Rating               0
Subscription Status         0
Payment Method              0
Shipping Type               0
Discount Applied            0
Promo Code Used             0
Previous Purchases          0
Preferred Payment Method    0
Frequency of Purchases      0
dtype: int64
```

Justification:

Cleaned the customer dataset by handling missing values, removing duplicates, standardizing city names, and encoding categorical variables using Pandas.

Task 2 – Sales Data Preprocessing

Task:
Preprocess **sales transaction data** using AI assistance.

Prompt:

Generate a Python script using Pandas to preprocess sales data. Convert date columns into datetime format, create month, year, and day-of-week features, normalize the sales amount column, and detect and handle outliers.

Code:

```
import pandas as pd

# Load dataset
data = pd.read_csv("sales_data.csv")

print("Original Dataset:")
print(data.head())

# Convert Sale_Date into datetime format
data['Sale_Date'] = pd.to_datetime(data['Sale_Date'])

# Create new date features
data['Month'] = data['Sale_Date'].dt.month
data['Year'] = data['Sale_Date'].dt.year
data['Day_of_Week'] = data['Sale_Date'].dt.dayofweek

# Normalize Sales Amount
sales_column = 'Sales_Amount'

data[sales_column] = data[sales_column].fillna(
    data[sales_column].median()
)
```

```
min_value = data[sales_column].min()
max_value = data[sales_column].max()

if max_value != min_value:
    data['Normalized_Sales'] = (
        (data[sales_column] - min_value) /
        (max_value - min_value)
    )
else:
    data['Normalized_Sales'] = 0

# Detect and remove outliers using IQR
Q1 = data[sales_column].quantile(0.25)
Q3 = data[sales_column].quantile(0.75)
IQR = Q3 - Q1

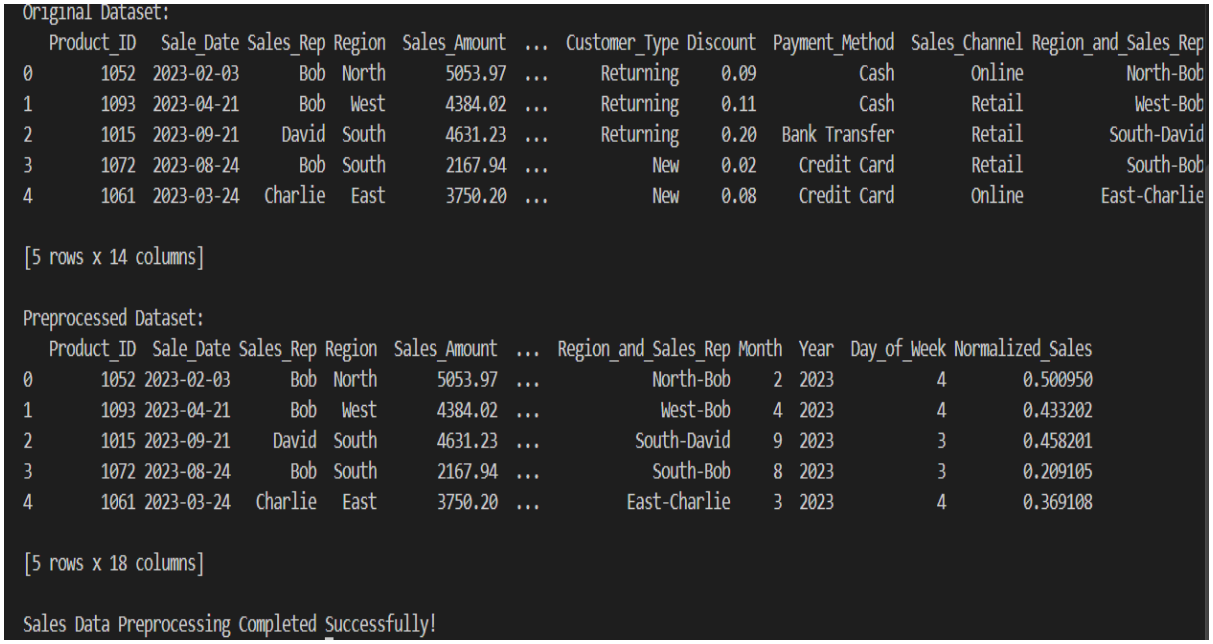
data = data[
    (data[sales_column] >= Q1 - 1.5 * IQR) &
    (data[sales_column] <= Q3 + 1.5 * IQR)
]

# Display preprocessed dataset
print("\nPreprocessed Dataset:")
print(data.head())

# Save processed dataset
data.to_csv("cleaned_sales_data.csv", index=False)
```

print("\nSales Data Preprocessing Completed Successfully!")

Output:



```
Original Dataset:
   Product_ID   Sale_Date Sales_Rep Region  Sales_Amount  ... Customer_Type Discount  Payment_Method  Sales_Channel Region_and_Sales_Rep
0        1052  2023-02-03       Bob  North       5053.97  ...     Returning     0.09            Cash         Online           North-Bob
1        1093  2023-04-21       Bob   West       4384.02  ...     Returning     0.11            Cash         Retail            West-Bob
2        1015  2023-09-21     David  South       4631.23  ...     Returning     0.20   Bank Transfer         Retail          South-David
3        1072  2023-08-24       Bob  South       2167.94  ...           New     0.02     Credit Card         Retail          South-Bob
4        1061  2023-03-24   Charlie   East       3750.20  ...           New     0.08     Credit Card         Online        East-Charlie

[5 rows x 14 columns]

Preprocessed Dataset:
   Product_ID  Sale_Date Sales_Rep Region  Sales_Amount  ... Region_and_Sales_Rep Month  Year  Day_of_Week Normalized_Sales
0        1052 2023-02-03       Bob  North       5053.97  ...           North-Bob     2  2023            4         0.500950
1        1093 2023-04-21       Bob   West       4384.02  ...            West-Bob     4  2023            4         0.433202
2        1015 2023-09-21     David  South       4631.23  ...          South-David     9  2023            3         0.458201
3        1072 2023-08-24       Bob  South       2167.94  ...          South-Bob     8  2023            3         0.209105
4        1061 2023-03-24   Charlie   East       3750.20  ...        East-Charlie     3  2023            4         0.369108

[5 rows x 18 columns]

Sales Data Preprocessing Completed Successfully!
```

Justification:

Preprocessed the sales dataset by converting dates into datetime format, creating month, year, and day-of-week features, normalizing sales amounts, and handling outliers using the IQR method.

Task 3 – Healthcare Data Preparation

Task:
Clean and preprocess a **healthcare dataset** using AI scripts.

Prompt:

Generate a Python script using Pandas and scikit-learn to prepare the healthcare dataset. Handle missing values in blood_pressure and cholesterol, encode categorical features, scale numerical features using Min-Max scaling, and split the dataset into training and testing sets.

Code:

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder, MinMaxScaler

from sklearn.model_selection import train_test_split


# Load dataset
```

```
data = pd.read_csv("dirty_v3_path.csv")

print("Original Dataset:")
print(data.head())

# Remove unnecessary noise columns
data = data.drop(columns=["random_notes", "noise_col"], errors="ignore")

# Handle missing values
for col in data.columns:
    if pd.api.types.is_numeric_dtype(data[col]):
        data[col] = data[col].fillna(data[col].median())
    else:
        if not data[col].mode().empty:
            data[col] = data[col].fillna(data[col].mode()[0])

# Encode categorical columns
encoder = LabelEncoder()

for col in data.select_dtypes(include=["object", "string"]).columns:
    data[col] = encoder.fit_transform(data[col].astype(str))

# Display dataset columns
print("\nDataset Columns:")
print(data.columns.tolist())

# Separate features and target
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
```

```
# Scale numerical features
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

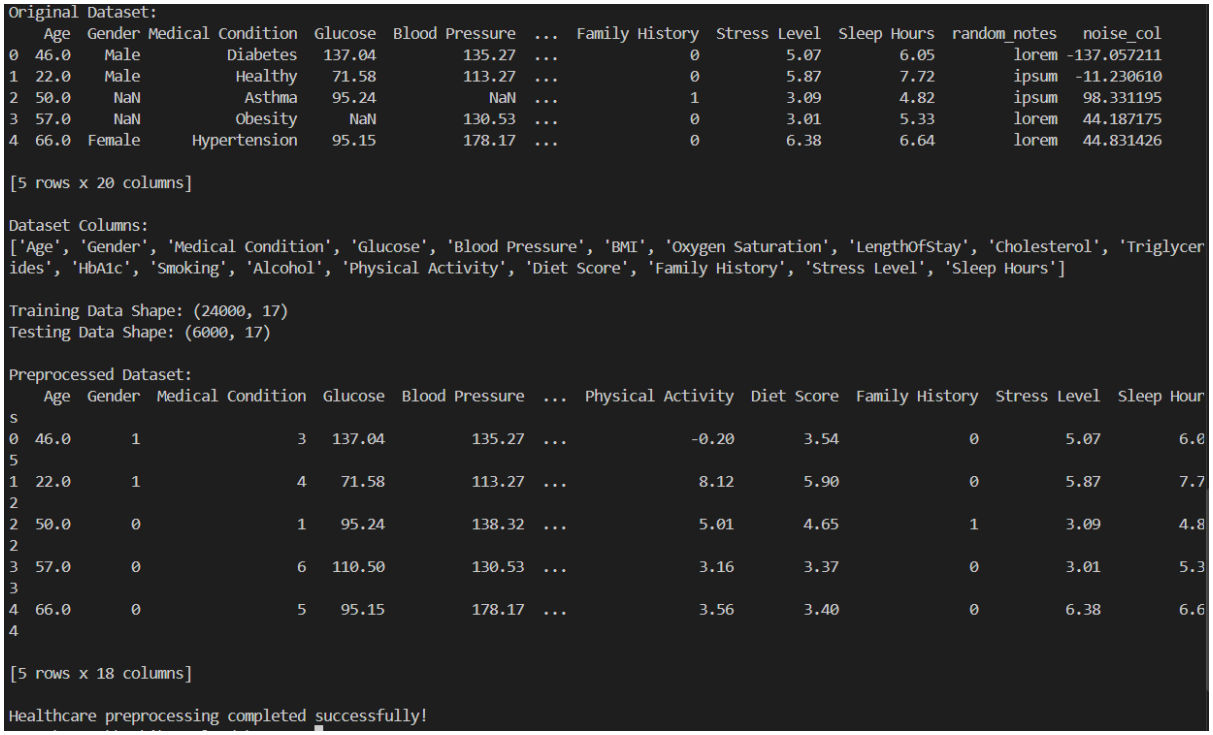
# Display results
print("\nTraining Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)

print("\nPreprocessed Dataset:")
print(data.head())

# Save cleaned dataset
data.to_csv("cleaned_healthcare.csv", index=False)

print("\nHealthcare preprocessing completed successfully!")
```

Output:



```
Original Dataset:
    Age  Gender Medical Condition  Glucose  Blood Pressure  ...  Family History  Stress Level  Sleep Hours  random_notes   noise_col
0  46.0    Male          Diabetes   137.04          135.27  ...               0          5.07         6.05         lorem -137.057211
1  22.0    Male           Healthy    71.58          113.27  ...               0          5.87         7.72         ipsum  -11.230610
2  50.0     NaN            Asthma    95.24             NaN  ...               1          3.09         4.82         ipsum   98.331195
3  57.0     NaN           Obesity      NaN          130.53  ...               0          3.01         5.33         lorem   44.187175
4  66.0  Female      Hypertension    95.15          178.17  ...               0          6.38         6.64         lorem   44.831426

[5 rows x 20 columns]

Dataset Columns:
['Age', 'Gender', 'Medical Condition', 'Glucose', 'Blood Pressure', 'BMI', 'Oxygen Saturation', 'LengthOfStay', 'Cholesterol', 'Triglycer
ides', 'HbA1c', 'Smoking', 'Alcohol', 'Physical Activity', 'Diet Score', 'Family History', 'Stress Level', 'Sleep Hours']

Training Data Shape: (24000, 17)
Testing Data Shape: (6000, 17)

Preprocessed Dataset:
    Age  Gender  Medical Condition  Glucose  Blood Pressure  ...  Physical Activity  Diet Score  Family History  Stress Level  Sleep Hour
s
0  46.0       1                  3   137.04          135.27  ...              -0.20        3.54               0          5.07         6.0
5
1  22.0       1                  4    71.58          113.27  ...               8.12        5.90               0          5.87         7.7
2
2  50.0       0                  1    95.24          138.32  ...               5.01        4.65               1          3.09         4.8
2
3  57.0       0                  6   110.50          130.53  ...               3.16        3.37               0          3.01         5.3
3
4  66.0       0                  5    95.15          178.17  ...               3.56        3.40               0          6.38         6.6
4

[5 rows x 18 columns]

Healthcare preprocessing completed successfully!
```

Justification:

Prepared the healthcare dataset by handling missing values, removing unnecessary noise columns, encoding categorical features, scaling numerical data using Min-Max scaling, and splitting the dataset into training and testing sets.

Task 4 – Employee Salary Data Preprocessing

Task:
Clean and preprocess an employee salary dataset using AI scripts.

Prompt:

Generate a Python script using Pandas and scikit-learn to preprocess employee salary data. Handle missing values, detect and remove salary outliers using the IQR method, convert department names to lowercase, and label encode job location and department columns. Display and save the cleaned dataset.

Code:

import pandas as pd

from sklearn.preprocessing import LabelEncoder

Load employee salary dataset

```
data = pd.read_csv("Employee_Salary_Dataset.csv")

print("Original Dataset:")
print(data.head())

# Handle missing values
for col in data.columns:
    if pd.api.types.is_numeric_dtype(data[col]):
        data[col] = data[col].fillna(data[col].median())
    else:
        data[col] = data[col].fillna(data[col].mode()[0])

# Standardize department names if available
if "Department" in data.columns:
    data["Department"] = data["Department"].str.lower().str.strip()

# Detect and remove salary outliers using IQR
Q1 = data["Salary"].quantile(0.25)
Q3 = data["Salary"].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

data = data[
    (data["Salary"] >= lower_limit) &
    (data["Salary"] <= upper_limit)
]
```

```
# Encode available categorical columns
encoder = LabelEncoder()

for col in ["Gender", "Job_Location", "Department"]:
    if col in data.columns:
        data[col] = encoder.fit_transform(data[col].astype(str))

# Display cleaned dataset
print("\nCleaned Dataset:")
print(data.head())

print("\nFinal Dataset Shape:", data.shape)

# Save cleaned dataset
data.to_csv("cleaned_employee_salary.csv", index=False)

print("\nEmployee salary preprocessing completed successfully!")
```

Output:

```
Original Dataset:
   ID  Experience_Years  Age  Gender  Salary
0   1                 5   28  Female  250000
1   2                 1   21    Male   50000
2   3                 3   23  Female  170000
3   4                 2   22    Male   25000
4   5                 1   17    Male   10000

Cleaned Dataset:
   ID  Experience_Years  Age  Gender  Salary
0   1                 5   28       0  250000
1   2                 1   21       1   50000
2   3                 3   23       0  170000
3   4                 2   22       1   25000
4   5                 1   17       1   10000

Final Dataset Shape: (33, 5)

Employee salary preprocessing completed successfully!
```

Justification:

Preprocessed the employee salary dataset by handling missing values, detecting and removing salary outliers using the IQR method, and encoding available categorical features such as gender. Saved the cleaned dataset for further analysis.